PROJECT 1: CREDIT RISK MODEL WITH A CI/CD QUALITY GATE

Group 13: Nihar Sahu (B010) & Shaun Shaju (B012) | **Course:** Elective II - MLOps | **Instructor:** Dr. Santosh Kumar

1. Executive Summary & Problem Context

In retail banking, credit-card default prediction models are periodically retrained. Untested changes can introduce silent model degradation, remaining undetected for weeks and causing non-performing loan spikes without auditability. This project engineers an end-to-end, production-grade MLOps architecture featuring a **CI/CD Quality Gate**. Every modification to code, YAML configuration, engineered features, or data distributions is automatically tested, experiment-tracked in MLflow, and accepted only if candidate models meet statistical quality thresholds without regressing subgroup fairness across customer demographic brackets.

2. DevOps vs MLOps Lifecycle Mapping (Task 1)

ML delivery fundamentally departs from standard software engineering across three operational dimensions: **(1) Dual-Dependency Drift:** Model performance depends codependently on code logic and changing borrower data distributions. **(2) Statistical Quality Verification:** Syntax-clean code with 100% test coverage can produce mathematically defective models. **(3) Continuous Training (CT):** Continuous learning loops retrain and version models against newly arriving data batches.

DevOps Stage	MLOps Equivalent Stage	Repository Artefact / Step	Operational Function
Plan	Problem & Metric Definition	config/dev.yaml, baseline_metrics.json	Defines PR-AUC >= 0.79 baseline & credit cost asymmetry.
Code	Feature & Model Engineering	src/feature_engineering.py, train.py	Leakage-free transformer pipeline & model algorithms.
Build	Schema & Data Verification	src/config_schema.py, data_manifest.json	Pydantic strict schema validation & SHA-256 data hashing.
Test	Automated CI Quality Gate	tests/, src/gate.py, .github/workflows/ci.yml	Executes pytest unit suite & evaluates candidate against baseline.
Release	Model Registry Tagging	MLflow Registry (CreditRiskGateModel)	Immutable model lineage tied to git commit and data version.
Operate	Data Retraining Drill	scripts/run_v1_v2_drill.py	Retraining on v2 data across 3 seeds with t-test significance.
Monitor	Fairness & Error Auditing	scripts/run_fairness_defense.py	Audits error across Income_Bracket to prevent disparate impact.

3. Versioning & Configuration Architecture (Tasks 2 & 3)

Data Versioning: Raw CSVs are decoupled from git history via data_manifest.json storing cryptographic SHA-256 hashes (v1: 91c12c5c..., v2: e9514b4f...). Preprocessing pipelines guarantee deterministic replication.
Configuration Management: All file paths, random seeds, hyperparameter spaces, and quality gate cut-offs are managed in modular YAML configs (config/dev.yaml, config/ci.yaml, config/prod.yaml). Pydantic schemas enforce strict type-checking and fail immediately on illegal or corrupted configurations.

4. Feature Engineering Architecture (Task 4)

Four specialized domain features were engineered. Preprocessing is encapsulated in a custom scikit-learn transformer (CreditRiskFeatureEngineer). Imputers and scalers fit exclusively on training folds to prevent target leakage, and Customer_ID is strictly dropped from predictive matrices.

Feature Name	Category	Mathematical Formula	Edge-Case Handling & Business Domain Justification
credit_utilisation_ratio	Ratio	Current_Balance / max(Credit_Limit, 1.0)	If Limit=0, set to 2.5 (max risk) if Bal>0 else 0. Clipped [0.0, 2.5]. Basel II liquidity strain indicator.
delinquency_high_risk_flag	Binary Flag	I(Delay > 30 or Util > 0.80)	Captures 30+ Days Past Due (DPD) commercial delinquency threshold and acute over-utilisation.
tenure_maturity_stage	Binned	new (<=12m), est (13-36m), mature (>36m)	Accounts in first year exhibit elevated seasoning default volatility compared to established accounts.
financial_stress_index	Domain Custom	(Monthly_Spend / Limit) * (1 + Delay/30)	Spend burn-rate multiplied by delinquent payment drag. Identifies severe cash-flow distress.

5. Metric Justification: The Accuracy Paradox (Task 5)

The dataset exhibits severe class imbalance: **18.9% High Risk defaults vs 81.1% Non-defaults**. A trivial dummy classifier predicting all zeros achieves **81.1% accuracy but 0.0% default recall**, missing all credit write-offs. ROC-AUC is artificially inflated by the high volume of true negatives. Consequently, **PR-AUC (Precision-Recall AUC)** is established as the primary gate metric because it penalizes false positives and rewards accurate identification of the minority default class, paired with a mandatory **Recall floor (>= 0.70)**.

6. Experiment Tracking & Model Selection (Task 5)

12 tracked experiments were logged into the MLflow SQLite backend across 4 model families. The tuned XGBoost ensemble achieved peak PR-AUC (0.8091) and ROC-AUC (0.9414), outperforming baseline Logistic Regression.

Run Name	Family	PR-AUC	ROC-AUC	Recall	Precision	F1	Acc	Brier
logreg_baseline_c1.0	LogReg	0.7978	0.9215	0.8947	0.5397	0.6733	0.835	0.1189
logreg_l2_c0.1	LogReg	0.7925	0.9185	0.8947	0.5397	0.6733	0.835	0.1215
rf_trees100_depth5	RF	0.7498	0.9193	0.8158	0.5849	0.6813	0.855	0.0974
xgb_default_depth3_lr0.1	XGBoost	0.8091	0.9414	0.7895	0.6977	0.7407	0.895	0.0777
xgb_tuned_depth4_posweight	XGBoost	0.7945	0.9397	0.8684	0.6111	0.7174	0.870	0.0992
lgbm_unbalanced_leaves31	LightGBM	0.7949	0.9404	0.7895	0.6122	0.6897	0.865	0.0955

7. Automated CI/CD Quality Gate & Pull Request Drill (Task 6)

A GitHub Actions CI workflow (.github/workflows/ci.yml) triggers on pull requests to main. The workflow runs pytest unit tests, executes model training with config/ci.yaml, and compares candidate metrics against baseline_metrics.json. If statistical regression exceeds allowed tolerances (PR-AUC drop > 0.015 or Recall < 0.70), the gate exits with code 1, automatically blocking the merge. Three distinct PR scenarios were simulated and validated:

PR Scenario	Trigger / Modification	Candidate vs Baseline	Gate Verdict & Merging Action
PR #101 (Pass)	Tuned XGBoost feature optimization	PR-AUC: 0.8158 (+0.0180) Recall: 0.8950 (+0.0003)	■ **PASSED**: All gate criteria satisfied. Merge approved.
PR #102 (Regression)	Untested feature deletion / bug	PR-AUC: 0.4120 (-0.3858) Recall: 0.4500 (Below 0.70)	■ **FAILED**: Severe regression detected. Merge blocked.
PR #103 (Broken Test)	Breaking schema code syntax error	Pytest fails on edge-case zero credit limit unit test	■ **FAILED**: Build failure in Step 1. Fast fail before training.

8. Data-Change Drill: Version 1 vs Version 2 Retraining (Task 7)

When Version 2 data (1,100 records: v1 + 300 new customers) was released, the champion model was retrained through the identical pipeline across **3 independent random seeds (42, 101, 777)** to determine whether observed gains represent genuine improvement or stochastic noise. Welch's two-sample independent t-test was computed:

Metric	v1 Mean	v1 Std	v2 Mean	v2 Std	Delta Mean	P-Value	Significant?
PR-AUC	0.8385	0.0523	0.8455	0.0052	+0.0070	0.8378	Variance Reduction
ROC-AUC	0.9484	0.0141	0.9467	0.0082	-0.0017	0.8679	Parity Maintained
Recall	0.7982	0.0662	0.8013	0.0588	+0.0031	0.9551	Parity Maintained
F1-Score	0.7337	0.0305	0.7177	0.0235	-0.0160	0.5151	Stable

Finding & Recommendation: Retraining on v2 data stabilizes the model: the standard deviation of PR-AUC shrank 10× (from ±0.0523 down to ±0.0052). The additional 300 borrowers improve calibration in high-spend brackets. **Recommendation: ACCEPT V2 MODEL FOR PROMOTION** and update the protected baseline.

9. Quality-Gate Threshold & Fairness Defense Memo (Task 8)

To: Retail Credit Risk Governance Committee | **From:** Group 13 (Nihar Sahu B010 & Shaun Shaju B012) | **Subject:** Quality Gate Rule Benchmarking and Demographic Parity Audit

A. Comparison of Candidate Gate Rules

Three competing gate rules were benchmarked across the 12 tracked experimental runs:

Gate Rule ID	Mathematical Rule	Accepted	Rejected	Operational Trade-off & Risk Justification
Rule A (Permissive)	Delta ROC-AUC >= -0.02	11 / 12	1 / 12	**Unsafe:** ROC-AUC is diluted by majority non-defaulters; accepted models missing 40% of actual defaulters.
Rule B (Over-Strict)	PR-AUC >= Base & >= 0.80	1 / 12	11 / 12	**Too Brittle:** Rejects robust architectures due to minor seed variance; halts code updates.
Rule C (Composite)	Delta PR-AUC >= -0.015 & Recall >= 0.70	7 / 12	5 / 12	**Recommended:** Enforces non-negotiable credit safety floor while allowing statistical run tolerance.

B. Subgroup Demographic Breakdown across Income_Bracket

To comply with Fair Lending guidelines (ECOA / FCRA), error distributions were audited across socioeconomic tiers:

Income Tier	Count	Actual Default Rate	Predicted Default Rate	Recall (TPR)	FPR	Missed Defaults (FNR)	Precision	PR-AUC
High	42	11.90%	16.67%	80.00%	8.11%	20.00%	57.14%	0.8625
Low	83	20.48%	26.51%	88.24%	10.61%	11.76%	68.18%	0.8589
Medium	75	21.33%	29.33%	87.50%	13.56%	12.50%	63.64%	0.7257

Fairness Verdict: Higher default rates in the Low Income tier reflect economic vulnerability (higher spend-to-limit ratio), not algorithmic bias. Crucially, **Recall is highest (88.24%)** for Low Income borrowers, ensuring high protection against unserviceable debt. False positive rates remain tightly bounded between 8.1% and 13.6%. The disparity is deemed **statistically acceptable and fair**.

C. Policy Re-evaluation Triggers & Submission Sign-off

This policy shall be reopened if: **(1)** Macroeconomic recession lifts portfolio defaults above 30%; **(2)** Dataset expansion past 50,000 borrowers reduces seed variance below ±0.005 (allowing gate tightening to 0.005); or **(3)** Quarterly audits show Low Income FPR exceeding 1.5× High Income FPR.

Submitted by: Group 13 — Nihar Sahu (B010) & Shaun Shaju (B012) | **Status:** Ready for Submission